

21/8/26

CS340

Lecture 10

Pumping Lemma for Regular Languages

Pumping Lemma is a simple tool to prove some languages to be non-regular.

Lemma

Let L be a regular language. There exists an integer $n > 0$ s.t. any $w \in L$ with $|w| \geq n$, may be written as $w = xyz$, where $|xy| \leq n$, $|y| \geq 1$ and for all $i \geq 0$, $xy^iz \in L$.

Before, we prove this Lemma, we would like to see how to use it.

Example: show that $L = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Solution Towards contradiction ~~let~~ assume L to be regular. Let n be the constant in pumping lemma.

Let ~~$w = xyz$~~ $w = 0^n 1^n$, $w \in L$.

By pumping lemma, there exist x, y, z s.t.

$w = xyz$. As $|xy| \leq n$, we know that

y consists of only 0's, also $|y| \geq 1$.

Consider $xy^0z = xz$ ($i=0$)

This string is of the form $0^m 1^n$, where ~~$m < n$~~ $m < n$
(we have lost at least one 0 by omitting y)

By defn of L , $xz \notin L$. However, by Pumping Lemma $xz \in L$. This gives the desired Contradiction. $\square$

Example

Let $L_{pr} = \{0^p \mid p > 2, p \text{ is a prime}\}$

We show that L_{pr} is not regular.

Again, towards contradiction assume that L_{pr} is regular and let n be the constant in pumping lemma.

Consider a prime $q \geq n+3$, $0^q \in L_{pr}$

By pumping lemma, $0^q = xyz$

and $xy^{q+1}z \in L \dots (i)$

$$|xy^{q+1}z| = |x| + (q+1)|y| + |z|$$

$$= q - \cancel{|y|} + (\cancel{q+1})|y| \quad (\because |x| + |z| = q - |y|)$$

$$= q(1 + |y|)$$

Ans $q(1+|y|)$ is not a prime,

$$xy^{q+1}z \notin L \quad (\text{By defn of } L)$$

This contradicts (i), the conclusion of PL (Pumping Lemma) $\square$

Exercise: Show that

$$L_{\text{perfect-sq}} = \{0^{n^2} \mid n \geq 0\} \text{ is not regular.}$$

Proof of Pumping Lemma

Let L be recognizable by a DFA $M = (Q, \Sigma, q_0, \delta, F)$.

Consider run of M on first n symbols of w .

$$q_0 a_1 q_1 a_2 q_2 \dots a_n q_n a_{n+1} q_{n+1} \dots a_{|w|} q_{|w|}$$

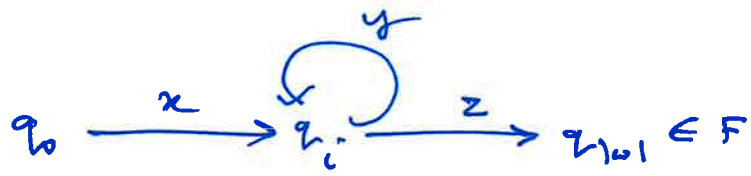
There are $n+1$ states $q_0, q_1, \dots, q_n$ in the prefix highlighted. As there are only n distinct states

in M , $q_i = q_j$ for some $i \neq j$, $0 \leq i, j \leq n$.

w.l.o.g. let $i < j$. We let $x = a_1 a_2 \dots a_i$

and $y = a_{i+1} a_{i+2} \dots a_j$. Pictorially

$$q_0 \xrightarrow{x} q_i \xrightarrow{y} \underset{\substack{q_i \\ = \\ q_j}}{q_i} \xrightarrow{z} q_{|w|} \in F.$$



So we have a loop on state q_i for string y .

Clearly, we can repeat y any number of times (including zero times), we always end up in $q_{|w|} \in F$.

Therefore $x y^i z \in L$ for $i \geq 0$.



Decision Procedures for Regular languages

For ~~a~~ regular languages, we can check emptiness algorithmically. That is, there is an algorithm which takes a ~~regular~~ regular language as an input and answers 'yes' if this language is empty and answers 'no' if it is not.

A question arises, how a regular language^{is} presented to our algorithm. It can be presented as DFA/NFA or R.E., we have seen effective (algorithmic) conversions from one form to another in previous lectures. We assume ~~that~~ here that it is presented as a transition diagram of DFA/NFA.

Our algorithm removes all labels from the transition diagrams (that is all input symbols). It now searches if there is a path from q_0 to one of the final states, that is if any final state is reachable from the ~~the~~ start state. This can be checked ~~by~~ by a DFS initiated from q_0 on the transition graph, and takes linear time (in size of the transition diagram). We summarized this in the theorem below.

Theorem

Emptiness of regular languages can be checked in linear time.

Similarly given a NFA, we can check if it accepts a finite or an infinite language. To do this, we first remove all states from the transition diagrams, from which no final state is reachable in zero or more transitions (zero transitions ensures that final states are not removed). This does not change the language accepted by the automaton. Now, the automaton accepts an infinite language iff there is a cycle in the transition diagram, which is reachable from the start state. This can easily be checked by a DFS. Overall this leads to $O(n^2)$ algorithm, because of the step of removing all states from which no final state is reachable. In summary, we have the following.

Theorem There is an $O(n^2)$ algorithm to check finiteness/infiniteness of regular languages.

An interesting corollary of the emptiness checking (and previous results on DFAs) is that there is an algorithm to check if two regular expressions define the same language.

Thm There is an algorithm which takes two R.E., r and s as input and checks if $L(r) = L(s)$.

Proof:

Construct automata M_r, M_s s.t. $L(r) = L(M_r)$ and $L(s) = L(M_s)$. Design automaton M to accept

$$(L(M_s) \cap \overline{L(M_r)}) \cup (L(M_r) \cap \overline{L(M_s)}).$$
 This can

be done as $\cup, \cap$ and $\overline{(\cdot)}$ are effective operations on DFAs. It is easy to see that $L(M) = \emptyset$ iff $L(M_r) = L(M_s)$. We check $L(M) = \emptyset$ using emptiness checking algorithm run on M . $\square$

Remark

Complexity of algorithm in the theorem above (checking equivalence of two ~~languages~~ ^{R.E.}) is exponential because constructing DFA from R.E. may take exponential time.